Vehicle Data Normalization with Retrieval-Augmented LLMs

Diego Quezada
Departamento de Informática
UTFSM
Valparaíso, Chile
diego.quezadac@sansano.usm.cl

Carlos Valle
Escuela de Ingeniería Informática
PUCV
Valparaíso, Chile
carlos.valle@pucv.cl

Ricardo Ñanculef
Departamento de Informática
UTFSM
Valparaíso, Chile
jnancu@inf.utfsm.cl

***Abstract*—Heterogeneous vehicle datasets across markets, languages, and platforms force every downstream ML pipeline to engineer dataset-specific normalization from scratch, a bottleneck repeatedly noted in the vehicle price prediction literature. Existing standards (VIN, schema.org Automotive, COVESA VSS) see little adoption in open data, leaving listings to diverge along naming, granularity, and scope axes. We present a vehicle data normalization framework that maps a free-text listing to an extensible normalized schema by chaining three steps: extract attribute values, retrieve candidates from self-growing per-attribute catalogs, and resolve matches at the record level. Our method achieves novelty F1 of 0.623 and 0.878 on brand-model pairs across 1,000 labeled listings from AutoScout24 (Germany) and MuCars (Morocco), respectively, with 95% bootstrap intervals above the extraction-only baseline. The utility of normalization extends downstream to vehicle price prediction, improving MAPE by 2.7–5.3% on MuCars across CatBoost, XGBoost, and LightGBM, and is larger on listings with canonicalized models, where MAE improves by up to 8.7%. Ablations show the record-level resolver achieves normalization quality and novelty-detection F1 unmatched by any cosine-similarity threshold tested, since no single cutoff can both merge synonyms and preserve novel entities.**

***Index Terms*—LLM, RAG, Information Extraction, Information Retrieval, Entity Resolution**

I. Introduction

The global used car market was valued at USD 1.90 trillion in 2024 and is projected to grow at a compound annual growth rate (CAGR) of 6.0% from 2025 to 2030 [1], [2]. Key factors driving this growth include rising new vehicle prices, enhanced vehicle financing options, and advancements in online platforms [2], [3], which facilitate access to inventory and information impacting consumer trust, such as online reviews and ratings [4].

The expansion of online platforms has produced a proliferation of heterogeneous vehicle datasets across automotive markets. Without a shared standard for representing vehicles, descriptions of the same real-world entity diverge along three axes: *naming inconsistency*, where the same concept appears under different strings (e.g., "mercedes", "mercedes-benz", "mercedes benz"); *granularity inconsistency*, where the same attribute is expressed at different levels of detail (e.g., "automatic" vs. "7-speed DCT"); and *scope inconsistency*, where a single field conflates values that belong to distinct real-world dimensions (e.g., a *fuel type* column containing both "petrol" and "phev"). Left unaddressed, these inconsistencies propagate through downstream machine learning (ML) pipelines, undermining feature extraction, similarity computation, and ultimately model performance.

Several standardization efforts have attempted to prescribe how a vehicle *should* be modeled, yet open datasets in practice do not follow any of them. The Vehicle Identification Number (VIN), standardized under ISO 3779 [5], is effective within closed industry systems and dealership networks, but its decoding tables are proprietary, vary across manufacturers, and VIN data are rarely present in open datasets. The NHTSA vPIC database [6] decodes VINs into approximately 130 attributes but is restricted to US-market vehicles and excludes condition, usage history, and equipment. The ACES/PIES standards [7] provide structured fitment data for aftermarket parts in North America, not whole-vehicle attributes. The W3C Automotive Ontology and its `schema.org` extension [8] define a minimal set of vehicle properties for web markup, but accept free-form text for key fields and omit trim, equipment, and condition entirely. COVESA's Vehicle Signal Specification (VSS) [9] standardizes in-vehicle telemetry signals rather than vehicle-as-product attributes, and its own gap analysis acknowledges persistent interoperability issues across OEMs [10]. A standard that is not adopted at scale offers no interoperability in practice: heterogeneous listings from open platforms continue to diverge along the three axes described above, regardless of which schema has been formally defined. This problem has been repeatedly noted in the vehicle price prediction literature [11]–[13], where studies report that extensive ad-hoc preprocessing is required for each new dataset and call for standardized representation protocols.

In this work, we propose a vehicle data normalization framework that combines large language models (LLMs) for information extraction (IE) and entity resolution (ER) with dense information retrieval (IR) over a knowledge catalog, following the retrieval-augmented generation (RAG) paradigm, to map any raw vehicle description to a normalized, domain-consistent representation suitable for downstream ML tasks. Our framework is designed to be open-source and extensible, allowing practitioners to define custom output schemas tailored

979-8-3195-1348-9/26/$31.00 ©2026 IEEE

to their specific attribute requirements.[1]

II. Related Work

A. Vehicle Data Preprocessing in ML

Existing ML studies on vehicle data typically build dataset-specific preprocessing pipelines that do not transfer across datasets, languages, or markets. Bergmann and Feuerriegel [14] mapped German equipment strings onto manually selected categories via keyword and regex rules co-designed with car dealers, describing the approach as "tailored" to their partner's market, while Fayyaz et al. [12] relied on regex extraction and per-field standardization to improve downstream regression on US listings and acknowledge their recipe is dataset-specific. Such specialization impairs transfer: Dutulescu et al. [11] trained on German listings and tested on Romanian ones, found that car-model embeddings failed to transfer for infrequent models, and called for "a method for dataset standardization across different car markets." The same limitation surfaces at scale, where Madhusudhanan et al. [13] found that the high cardinality of unstandardized categorical features prevented state-of-the-art tabular models from processing their ∼2M-record corpus.

B. Information Extraction

IE from product descriptions has evolved from task-specific sequence taggers to general-purpose LLM-based extractors. OpenTag [15] formalized product attribute value extraction (AVE) as sequence tagging with a BiLSTM-CRF, and Wang et al. [16] later reformulated AVE as question answering over a shared BERT encoder. The closest line of work to ours is that of Brinkmann et al. Their initial study [17] found that ChatGPT performs comparably to fine-tuned pre-trained language models (PLMs) on product IE while needing far less training data, and ExtractGPT [18] extended this comparison to GPT-3.5, GPT-4, and Llama-3-70B. Most relevant of all, they broadened AVE to include *normalization* [19] and found that LLMs are especially strong at name expansion and string wrangling, precisely the capability our framework exploits. In a complementary direction, GoLLIE [20] showed that encoding annotation guidelines as Python class definitions with docstrings enables high-quality zero-shot IE, a role analogous to that of the structured vehicle schema in our approach.

C. Information Retrieval

The RAG paradigm [21] combines parametric and non-parametric knowledge by coupling a seq2seq model with a dense vector index. For the retrieval component itself, Karpukhin et al. [22] established that learned dual-encoder embeddings (Dense Passage Retrieval) outperform BM25, and Izacard et al. [23] later showed that fully unsupervised dense retrieval via contrastive learning matches BM25 without labeled query-passage pairs, which is relevant when labeled vehicle matching data are scarce. Two further ingredients make retrieval efficient. On the indexing side, the FAISS library [24], [25] provides schemes such as HNSW; on the embedding side, Sentence-BERT [26] established pre-computed sentence embeddings for similarity search, and recent models such as E5 [27] and BGE [28] lead retrieval benchmarks.

D. Entity Resolution

ER determines whether two surface forms refer to the same real-world entity (e.g., "mercedes" versus "mercedes-benz"). Pre-LLM approaches fine-tuned PLMs: Ditto [29] adapted BERT and RoBERTa with domain-knowledge injection and data augmentation, while Narayan et al. [30] were the first to cast entity matching as a prompting task for GPT-3. The most systematic investigation of LLM-based ER comes from the Mannheim group. Peeters and Bizer [31] gave the first evaluation of ChatGPT for entity matching and found it competitive with fine-tuned RoBERTa, and their follow-up MatchGPT [32] reported that zero-shot GPT-4 outperforms fine-tuned PLMs on previously unseen entities, a setting directly analogous to vehicle normalization, where new brands, models, and trims appear continuously. Later work weighs the trade-offs. Steiner et al. [33] found that fine-tuning helps smaller models but can hurt cross-domain transfer for larger ones, and on the efficiency side, Zhang et al. [34] showed a fine-tuned GPT-2 nears MatchGPT quality with far fewer parameters while Wang et al. [35] found a selecting-from-candidates strategy most cost-effective.

III. Method

We frame vehicle normalization as a three-component pipeline (Fig. 1). A raw record $\mathbf{x}$ is first serialized and passed to an LLM that jointly extracts candidate attribute values $\hat{\mathbf{x}} = (\hat{a}_1, \ldots, \hat{a}_m)$. Closed-domain attributes are emitted directly. Each open-domain attribute $\hat{a}_j$ is encoded and matched against a knowledge catalog $\mathcal{K}_j$ to retrieve its top-k canonical candidates $\mathcal{R}_j$. Finally, an LLM resolver decides whether any candidate $c_i \in \mathcal{R}_j$ refers to the same real-world entity as $\hat{a}_j$. If one does, it replaces $\hat{a}_j$, otherwise $\hat{a}_j$ is kept as a novel entity and appended to $\mathcal{K}_j$. The remainder of this section formalizes each component.

Let the normalized vehicle schema consist of m attributes $A_1, A_2, \ldots, A_m$, where each attribute A_j is associated with a domain $\mathcal{V}_j$ of valid values. Attribute domains are partitioned into two types:

$$\mathcal{V}_j = \begin{cases} \mathcal{V}_j^{\text{closed}} & \text{if } j \in \mathcal{I}_{\text{closed}}, \\ \mathcal{V}_j^{\text{open}} & \text{if } j \in \mathcal{I}_{\text{open}}. \end{cases} \tag{1}$$

A *closed-domain* attribute has a domain that is fully specified in advance and does not change during normalization. This includes both finite categorical sets and numerical ranges (e.g., mileage $\in \mathbb{R}^+$). An *open-domain* attribute (e.g., brand, model) has a domain that grows as the system encounters previously unseen entities.

The normalized vehicle space is the Cartesian product of all attribute domains:

$$\tilde{\mathcal{X}} = \prod_{j=1}^{m} \mathcal{V}_j. \tag{2}$$

[1] Source code is available at https://github.com/diegoquezadac/vn.

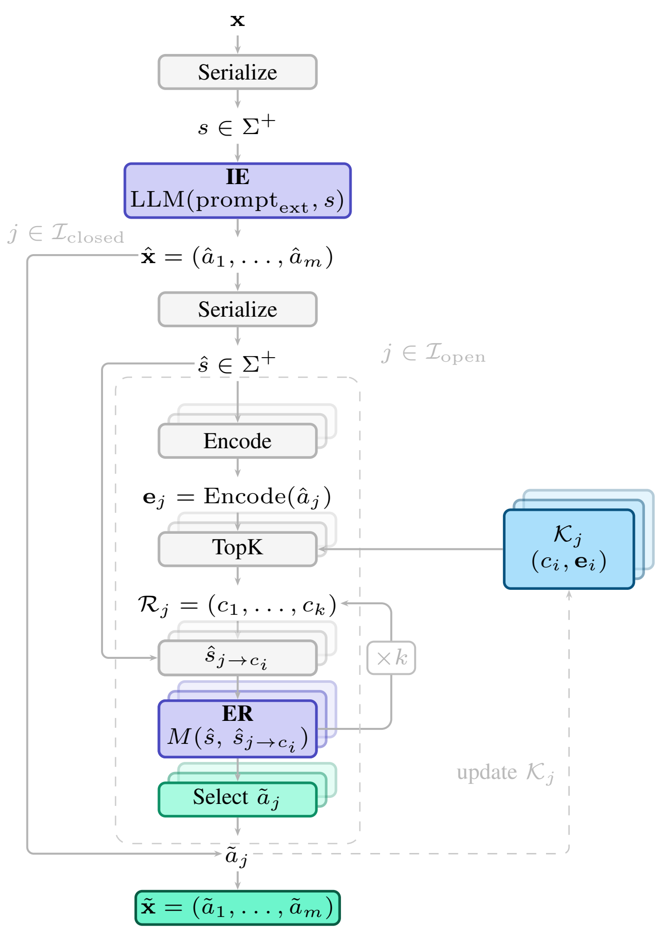

Fig. 1. Vehicle normalization pipeline. A raw record $\mathbf{x}$ is serialized and fed to an LLM that jointly extracts candidate attribute values $\hat{\mathbf{x}}$ (IE). Closed-domain attributes ($\mathcal{I}_{\text{closed}}$) bypass the loop and are emitted as-is. Each open-domain attribute ($\mathcal{I}_{\text{open}}$, dashed box) is encoded into $\mathbf{e}_j$, its top-k canonical candidates $\mathcal{R}_j$ are retrieved from catalog $\mathcal{K}_j$ via HNSW, and each candidate c_i is substituted into $\hat{s}_{j\to c_i}$ and tested by an LLM entity resolver $M(\cdot,\cdot)$. Stacked blocks represent one instance per open-domain attribute, and the $\times k$ arrow indicates that for each retrieved value we repeat the substitution-and-resolution process within each block. Select returns the first matching c_{i^*}; otherwise $\hat{a}_j$ is kept as a novel entity and the catalog is expanded (dashed feedback). See Algorithm 1.

Vehicle normalization is formulated as a mapping

$$\phi : \Sigma^+ \to \tilde{\mathcal{X}}, \tag{3}$$

that takes a serialized vehicle representation $s = \mathrm{Serialize}(\mathbf{x}) \in \Sigma^+$, where $\mathbf{x}$ is the raw input record, Σ denotes the character alphabet, and $\mathrm{Serialize} : \mathcal{X} \to \Sigma^+$ renders a vehicle representation as a string. The mapping produces $\phi(s) = \tilde{\mathbf{x}} \in \tilde{\mathcal{X}}$.

a) Knowledge catalog: For each open-domain attribute A_j with $j \in \mathcal{I}_{\text{open}}$, we maintain a knowledge catalog:

$$\mathcal{K}_j = \{(c_1, \mathbf{e}_1), \ldots, (c_{|\mathcal{K}_j|}, \mathbf{e}_{|\mathcal{K}_j|})\}, \tag{4}$$

where each pair $(c_i, \mathbf{e}_i)$ consists of a canonical entity value $c_i \in \mathcal{V}_j^{\text{open}}$ and its embedding $\mathbf{e}_i = \mathrm{Encode}(c_i) \in \mathbb{R}^d$. Any text encoder may serve as the encoding function.

b) Information extraction: Given a serialized vehicle description s, a single LLM call extracts candidate values for all attributes simultaneously:

$$\hat{\mathbf{x}} = (\hat{a}_1, \ldots, \hat{a}_m) = \mathrm{LLM}(\mathrm{prompt}_{\text{ext}}, s). \tag{5}$$

For closed-domain attributes, the extracted value is used directly as $\tilde{a}_j = \hat{a}_j$, since the prompt constrains the LLM output to $\mathcal{V}_j^{\text{closed}}$. For open-domain attributes, the candidate $\hat{a}_j$ is passed to the retrieval and resolution steps described below.

c) Information retrieval: For each open-domain attribute A_j, the extracted candidate is embedded as $\mathbf{e}_j = \mathrm{Encode}(\hat{a}_j)$, and the top-$k$ most similar canonical entities are retrieved from catalog $\mathcal{K}_j$:

$$\mathcal{R}_j = \text{top-k}_{(c_i,\mathbf{e}_i)\in\mathcal{K}_j}\ \mathrm{sim}(\mathbf{e}_j, \mathbf{e}_i), \quad \mathrm{sim}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u}^\top \mathbf{v}}{\|\mathbf{u}\|_2\, \|\mathbf{v}\|_2}. \tag{6}$$

The candidates in $\mathcal{R}_j$ are ranked in decreasing order of similarity.

d) Entity resolution: Given the extracted attributes $\hat{\mathbf{x}}$ and the ranked candidates $\mathcal{R}_j = (c_1, c_2, \ldots, c_k)$ for each open-domain attribute A_j, we compare two serialized vehicle descriptions. Let $\hat{s} = \mathrm{Serialize}(\hat{\mathbf{x}})$ be the serialization of the extracted attributes. For each candidate $c_i \in \mathcal{R}_j$, we construct a variant in which the value of attribute A_j is replaced by c_i:

$$\hat{s}_{j\to c_i} = \mathrm{Serialize}(\hat{a}_1, \ldots, c_i, \ldots, \hat{a}_m). \tag{7}$$

The LLM then evaluates whether both descriptions refer to the same vehicle:

$$M(\hat{s},\ \hat{s}_{j\to c_i}) = \mathrm{LLM}(\mathrm{prompt}_{\text{res}},\ \hat{s},\ \hat{s}_{j\to c_i}) \in \{0, 1\}. \tag{8}$$

All k candidates are evaluated and the normalized value is:

$$\tilde{a}_j = \begin{cases} c_{i^*}, & i^* = \min\{i : m_i = 1\}, \\ \hat{a}_j & \text{otherwise (novel entity)}, \end{cases} \tag{9}$$

where $m_i = M(\hat{s},\ \hat{s}_{j\to c_i})$. In the novel-entity case, the catalog is expanded: $\mathcal{K}_j \leftarrow \mathcal{K}_j \cup \{(\hat{a}_j, \mathrm{Encode}(\hat{a}_j))\}$.

Algorithm 1 summarizes the complete pipeline.

IV. Experiments

We evaluate the pipeline end-to-end by comparing its normalized output to human-annotated ground truth on real listings.

A. Experimental Setup

a) Schema: The normalized schema comprises $m = 26$ attributes: brand, model, year, body type, transmission type, transmission technology, transmission gears, energy source, propulsion system, fuel type, drive type, engine aspiration, injection type, cylinders, condition, color, doors, and equipment, together with four numeric attributes (mileage, engine size, engine power, fuel consumption) each paired with a unit field. Brand and model are open-domain; the remaining attributes are closed-domain, with domains given in Table I. The full schema with per-attribute descriptions accompanies our released code.

Algorithm 1 Vehicle Normalization.

Require: Raw vehicle record $\mathbf{x}$, Catalogs $\{\mathcal{K}_j\}_{j\in\mathcal{I}_{\text{open}}}$
Ensure: Normalized representation $\tilde{\mathbf{x}} = (\tilde{a}_1, \dots, \tilde{a}_m)$
$s \leftarrow \text{Serialize}(\mathbf{x})$
$\hat{\mathbf{x}} \leftarrow \text{LLM}(\text{prompt}_{\text{ext}}, s)$
for $j \in \mathcal{I}_{\text{closed}}$ **do**
  $\tilde{a}_j \leftarrow \hat{a}_j$
end for
$\hat{s} \leftarrow \text{Serialize}(\hat{\mathbf{x}})$
for $j \in \mathcal{I}_{\text{open}}$ **do**
  $\mathbf{e}_j \leftarrow \text{Encode}(\hat{a}_j)$
  $\mathcal{R}_j \leftarrow \text{TopK}_{(c_i,\mathbf{e}_i)\in\mathcal{K}_j} \text{sim}(\mathbf{e}_j, \mathbf{e}_i)$
  for $c_i \in \mathcal{R}_j$ in ranked order **do**
    $\hat{s}_{j\to c_i} \leftarrow \text{Serialize}(\hat{a}_1, \dots, c_i, \dots, \hat{a}_m)$
    $m_i \leftarrow M(\hat{s}, \hat{s}_{j\to c_i})$
  end for
  if $\exists\, i : m_i = 1$ **then**
    $i^* \leftarrow \min\{i : m_i = 1\}$
    $\tilde{a}_j \leftarrow c_{i^*}$
  else
    $\tilde{a}_j \leftarrow \hat{a}_j$ ▷ novel entity
    $\mathcal{K}_j \leftarrow \mathcal{K}_j \cup \{(\hat{a}_j, \text{Encode}(\hat{a}_j))\}$
  end if
end for
return $\tilde{\mathbf{x}} = (\tilde{a}_1, \dots, \tilde{a}_m)$

TABLE I
DOMAIN OF CLOSED-DOMAIN CATEGORICAL ATTRIBUTES

Attribute	Domain
Body type	sedan, hatchback, suv, crossover, coupe, convertible, wagon, pickup, van, minivan, roadster, other
Drive type	fwd, rwd, awd, 4wd, other
Transmission type	manual, automatic, semi-automatic
Transmission tech.	tc, cvt, sct, dct, other
Energy source	fossil, hybrid, electric, other
Propulsion system	icev, ev, pev, bev, hev, mhv, phev, erev, fcev, pfcev, other
Fuel type	petrol, diesel, cng, lpg, lng, methanol, propane, hydrogen, other
Engine aspiration	natural, turbo, supercharger, other
Injection type	direct, indirect, dual, other
Mileage unit	km, mi
Engine size unit	l, cc
Engine power unit	kw, ps, hp
Fuel cons. unit	l/100km, mpg, km/l
Condition	new, excellent, very good, good, fair, poor, salvage
Color	black, white, silver, grey, red, blue, green, yellow, orange, brown, beige, gold, purple, other

b) Models: Information extraction uses OpenAI's *gpt-4.1-nano* and entity resolution uses OpenAI's *gpt-4.1-mini*. Candidate embeddings for open-domain attributes are computed with OpenAI's *text-embedding-3-small*.

c) Retrieval configuration: Open-domain attributes are indexed with FAISS HNSW ($M = 32$, $ef_{\text{search}} = 64$, $k = 5$).

d) Serialization: $\text{Serialize}(\mathbf{x})$ concatenates column names with their values (e.g., *col1: val1, col2: val2, …*). The same operator renders extracted representations $\hat{\mathbf{x}}$ and candidate replacements $(\hat{a}_1, \dots, c_i, \dots, \hat{a}_m)$ in the entity-resolution step.

e) Baselines: We compare the full pipeline (IE+IR+ER) against two baselines (Fig. 2). IE sets $\tilde{a}_j = \hat{a}_j$ for all j, isolating the contribution of retrieval and resolution. IE+IR keeps $\tilde{a}_j = \hat{a}_j$ for $j \in \mathcal{I}_{\text{closed}}$ and, for $j \in \mathcal{I}_{\text{open}}$, sets $\tilde{a}_j = c_1$ when $\text{sim}(\mathbf{e}_j, \mathbf{e}_{c_1}) \geq \tau$, for a cosine-similarity threshold τ, and $\tilde{a}_j = \hat{a}_j$ otherwise, isolating the contribution of the LLM resolver M.

B. Datasets

Our experiments draw on three publicly available automotive listing datasets covering distinct geographic markets, schema conventions, and languages, summarized in Table II. DVM-CAR [36] is a UK dataset collected from online classified advertising platforms. MuCars [37], [38] is a Moroccan dataset with several market-specific attributes, including Fiscal Power as a tax-based engine category. AutoScout24 [39] is a German dataset with fully structured fields.

TABLE II
DATASET STATISTICS

Dataset	Listings	Brands	Models	#Attrs.	Country
DVM-CAR	268,255	101	1,011	16	UK
MuCars	101,896	81	1,049	15	Morocco
AutoScout24	251,079	47	1,312	15	Germany

We instantiate the catalog $\mathcal{K}_j$ for the open-domain attributes from DVM-CAR, which has the broadest brand coverage in our pool. Because DVM-CAR seeds the catalog, evaluation is restricted to MuCars and AutoScout24, where normalization is non-trivial.

C. Evaluation Data

a) Record sampling: From MuCars and AutoScout24 we draw 500 listings each via proportional stratified sampling on brand, yielding 1000 evaluation records. Brand is the dominant source of attribute heterogeneity across listings, and single-dimensional stratification keeps all strata populated at this sample size. Proportional allocation preserves the population brand distribution in expectation, so dataset-level metrics estimated on the sample remain unbiased estimators of the corresponding population mean.

As a descriptive diagnostic, we report the fraction of rows whose brand is in $\mathcal{K}_{\text{brand}}$ and whose model is in $\mathcal{K}_{\text{model}}$, both after lowercasing and whitespace stripping, in the full population and in the realized sample. Model membership is evaluated at

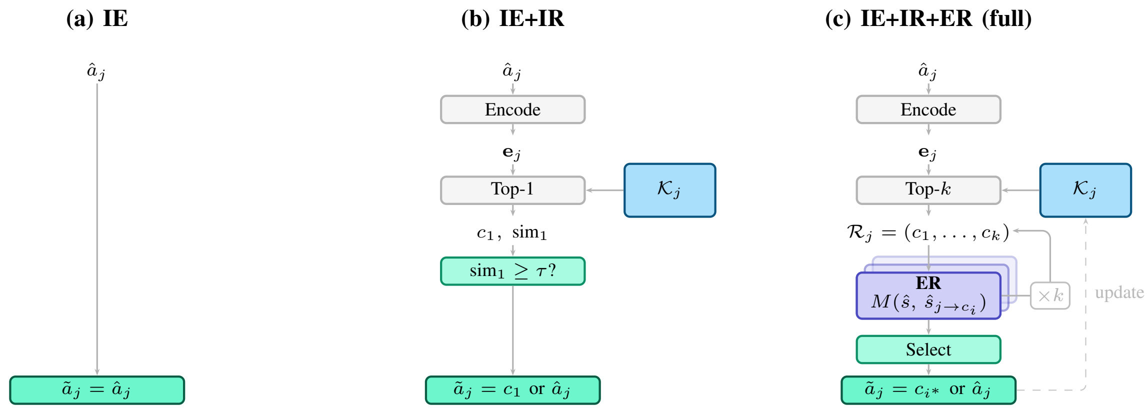

Fig. 2. Configurations compared: IE, IE+IR (top-1 if cosine $\geq \tau$), and IE+IR+ER (LLM resolver over top-k).

the pair level: a model is considered in $\mathcal{K}_{\text{model}}$ only if the specific (brand, model) combination appears in the catalog, which avoids spurious hits from identically named models across different brands. Model strings prefixed with the brand (common in AutoScout24) have the prefix stripped before lookup, approximating what the IE step extracts at inference time. Brand coverage is near-complete (population/sample 98.0%/98.0% on AutoScout24 and 98.9%/99.2% on MuCars), whereas model coverage is markedly lower (65.4%/68.0% and 63.6%/67.8% respectively); the realized sample closely tracks the population in every case.

b) Human annotation: Three trained annotators independently label every record using a structured form backed by per-attribute data validation. All three annotators hold bachelor's degrees, have prior experience in the automotive industry, and contributed as volunteer collaborators. For each record, the annotator reads the serialized listing and supplies a value for each of the 26 attributes, drawing from the closed-domain value lists or the DVM-CAR catalog where applicable. When the correct brand or model is not in the catalog, the annotator enters the value as written in the listing and flags it as `novel`. Attributes not mentioned in the listing are left blank. Each annotator completed a 20-record pilot round, after which instructions were revised to resolve systematic disagreements before the full annotation run.

c) Ground-truth construction: The ground-truth value for each (record, attribute) cell is the majority vote across the three annotators. Cells on which all three annotators disagree are marked `ABSTAIN` and excluded from headline metrics but retained for qualitative analysis. Inter-annotator agreement is measured with Fleiss' κ [40] per attribute and pooled, and with averaged pairwise raw agreement P_A as a secondary indicator. Attributes falling below $\kappa = 0.60$ are flagged and analyzed separately from the headline metrics [41]. Table III reports inter-annotator agreement per dataset.

The reported attribute count falls below the schema's 26 because the schema is general by design, whereas any individual dataset describes vehicles at its own level of detail and populates only a subset of the attributes. When a listing contains no information about an attribute, annotators leave the

TABLE III
INTER-ANNOTATOR AGREEMENT ON ATTRIBUTES WITH $\geq 10\%$ NON-NULL COVERAGE; **#LOW-κ**: $\kappa < 0.60$.

Dataset	#Attrs.	Pooled κ	Pooled P_A	#low-κ
AutoScout24	12	0.852	0.949	2
MuCars	12	0.591	0.864	4

cell blank, producing columns that are near-constant across the sample and on which Fleiss' κ is not statistically interpretable. We therefore restrict reporting to attributes with at least 10% non-null coverage per annotator. This is a property of the source data, not of the schema itself: Section IV-G shows that the same 26-attribute schema can be populated in full when the LLM is prompted to infer values from contextual cues (e.g., body type inferred from a model name), drawing on knowledge that lies outside the annotator's literal reading of the listing.

The MuCars pooled $\kappa = 0.591$ is a coverage artifact rather than a signal of annotator disagreement. Four attributes (*Mileage unit*, *Body type*, *Energy source*, *Propulsion system*) are blank in 70–88% of records, leaving annotators to choose between marking the field unobservable or inferring a market default (the implicit *km* unit; *fossil* or *icev* for older ICE-dominated listings); this policy split, not disagreement on extracted content, accounts for the negative per-attribute κ on those four. On the eight MuCars attributes that listings actually populate, per-attribute κ ranges from 0.962 to 1.000; on AutoScout24's ten populated attributes, from 0.918 to 1.000. Pooled κ averaged over a schema with several near-empty attributes therefore underestimates annotation reliability [42].

D. Metrics

We evaluate the pipeline along three dimensions: normalization quality, novelty detection, and catalog compression. All three are compared across three variants: the extraction-only baseline (IE), a retrieval-with-threshold variant that replaces the entity-resolution step by a top-1 cosine match above threshold τ (IE+IR), and the full pipeline (IE+IR+ER).

a) Normalization: For each open-domain attribute A_j we compare the normalized value $\tilde{a}_j$ produced by the pipeline to the human-annotated ground-truth value $\tilde{a}_j^\star$ on a per-record basis. We report two quantities. First, accuracy, the fraction of records for which $\tilde{a}_j = \tilde{a}_j^\star$ after case folding and whitespace collapse. Second, the macro-averaged F1 across the set of canonical classes appearing in either the predictions or the ground truth: precision, recall, and F1 are computed per class, treating membership in that canonical value as a binary decision, and then averaged with equal weight per class. Macro-F1 is reported alongside accuracy because the class distribution is long-tailed. Accuracy is dominated by a few popular models, whereas macro-F1 weights rare canonical values equally and therefore tracks the tail. Records for which all annotators disagreed are marked `ABSTAIN` and excluded.

b) Novelty detection: For each record we form the pair $(\mathrm{brand}, \mathrm{model})$ from both the ground truth and the pipeline output. A pair is declared *novel* if it is absent from the catalog; this yields a binary classification on each record, one label per side. We report precision, recall, and F1 on the novel class. Precision is the operationally important metric, since low precision floods the catalog with near-duplicates, whereas recall tracks whether genuinely new entities are being caught.

c) Compression: For each open-domain attribute we count the number of unique values emitted by the extraction-only baseline, $|U_{\mathrm{IE}}|$, and the number of unique canonical values produced by the full pipeline, $|U_{\mathrm{VN}}|$, on the same set of records. Both vocabularies are counted after lowercasing and whitespace stripping, so the reported compression reflects merges of genuinely distinct surface forms rather than trivial casing or whitespace differences. We report the compression percentage $1 - |U_{\mathrm{VN}}|/|U_{\mathrm{IE}}|$, which is the fraction of surface variants that the pipeline has merged into a single canonical entity (for example, "mercedes", "mercedes-benz", and "benz" collapsing to a single entry). Positive compression indicates that the canonicalization step is actively resolving naming inconsistency, whereas zero compression indicates that the extractor already yields canonical strings or that no merges were performed on this sample.

E. Results

Table IV reports pipeline evaluation on AutoScout24 and MuCars across the three dimensions defined in Section IV-D. Table V complements this with per-attribute extraction accuracy on attributes with reliable annotator agreement. Throughout the discussion, IE+IR uses cosine threshold $\tau = 0.80$, with a sensitivity study over τ deferred to Section IV-F. All metrics are computed on $n = 500$ evaluation records per dataset, and 95% bootstrap confidence intervals (1,000 resamples) are reported in square brackets alongside headline metrics.

a) Normalization: Brand accuracy saturates at 1.000 once any retrieval step is introduced, reflecting that the DVM-CAR brand catalog covers nearly every brand in both datasets. We therefore focus on the *model* attribute, where the normalization problem is genuinely non-trivial. Adding retrieval (IE+IR) improves model accuracy over the IE baseline by roughly four percentage points on each dataset, from 0.783 to 0.821 on AutoScout24 and from 0.862 to 0.908 on MuCars. The full pipeline reaches 0.865 and 0.930 respectively. The 95% bootstrap intervals for the full pipeline ($[0.835, 0.894]$ on AutoScout24 and $[0.906, 0.950]$ on MuCars) lie strictly above the corresponding IE intervals ($[0.744, 0.819]$ and $[0.830, 0.888]$), so the improvement over the IE-only baseline is statistically clean on both datasets. Macro-F1 follows the same ordering, rising from 0.822 $[0.792, 0.842]$ to 0.890 $[0.866, 0.912]$ on AutoScout24 and from 0.832 $[0.804, 0.861]$ to 0.885 $[0.869, 0.921]$ on MuCars, indicating that the accuracy improvement is not driven purely by popular classes. Against the threshold variant, the full pipeline's accuracy edge is directional rather than statistically sharp: at $n = 500$ per dataset the confidence intervals of the two overlap on model accuracy.

b) Novelty detection: The largest qualitative gap between variants appears on novelty detection, the task of flagging vehicle models absent from the DVM-CAR catalog. The full pipeline improves novelty precision substantially on both datasets: from 0.520 $[0.426, 0.607]$ to 0.793 $[0.696, 0.878]$ on MuCars, and from 0.280 $[0.213, 0.355]$ to 0.463 $[0.366, 0.573]$ on AutoScout24. Novelty F1 rises from 0.684 $[0.597, 0.755]$ to 0.878 $[0.812, 0.932]$ on MuCars and from 0.437 $[0.352, 0.524]$ to 0.623 $[0.525, 0.715]$ on AutoScout24. The 95% bootstrap intervals indicate that these improvements are robust under resampling, while recall stays at or above 0.95 across variants. The threshold variant yields novelty F1 of 0.504 $[0.399, 0.597]$ on AutoScout24 and 0.795 $[0.725, 0.854]$ on MuCars, both below the full pipeline's 0.623 and 0.878. The τ-ablation (Section IV-F1) confirms that no fixed cutoff matches the full pipeline jointly on model and novelty quality. This is consistent with the design intent of the ER step: when the vector-space neighborhood is genuinely ambiguous, a fixed cutoff cannot distinguish legitimate synonyms from new entities, and deferring the decision to the LLM recovers both precision and compression.

c) Compression: The full pipeline compresses the model vocabulary from 238 to 228 unique values on AutoScout24 (4.2%) and from 196 to 185 on MuCars (5.6%), confirming that the canonicalization step actively merges surface-form duplicates into shared canonical entries. The threshold variant compresses by 7.5% on AutoScout24 and 4.1% on MuCars; on AutoScout24 this exceeds the full pipeline numerically, but as Section IV-F1 shows, lower τ inflates compression by collapsing some genuine novelties, while the full pipeline's compression concentrates on correct merges.

d) Per-attribute extraction accuracy: Table V reports accuracy on every attribute for which annotator agreement meets the reliability threshold ($\kappa \geq 0.60$). The LLM extractor reaches macro-averaged accuracy of 0.983 on AutoScout24 (10 attributes) and 0.979 on MuCars (7 attributes, mileage excluded because MuCars encodes this field as numeric ranges rather than point values, incompatible with the scalar annotation schema). Most closed-domain attributes saturate at

TABLE IV
EVALUATION RESULTS ON AUTOSCOUT24 AND MUCARS: NORMALIZATION, NOVELTY DETECTION, AND CATALOG COMPRESSION.

Variant	AutoScout24								MuCars							
	Brand		**Model**		**Novelty**			**Compr.**	**Brand**		**Model**		**Novelty**			**Compr.**
	Acc	*F1*	*Acc*	*F1*	*F1*	*P*	*R*		*Acc*	*F1*	*Acc*	*F1*	*F1*	*P*	*R*	
IE	0.992	0.917	0.783	0.822	0.437	0.280	**1.000**	n/a	1.000	1.000	0.862	0.832	0.684	0.520	**1.000**	n/a
IE+IR ($\tau = 0.80$)	1.000	1.000	0.821	0.838	0.504	0.354	0.875	7.5%	1.000	1.000	0.908	0.852	0.795	0.725	0.879	4.1%
IE+IR+ER (full)	1.000	1.000	**0.865**	**0.890**	**0.623**	**0.463**	0.950	**4.2%**	1.000	1.000	**0.930**	**0.885**	**0.878**	**0.793**	0.985	**5.6%**

1.000, reflecting that the extraction prompt constrains outputs to $\mathcal{V}_j^{\text{closed}}$. Attributes where the pipeline adds measurable value over IE are concentrated on brand and model, where retrieval and LLM resolution improve accuracy beyond what extraction alone yields.

TABLE V
PER-ATTRIBUTE EXTRACTION ACCURACY. IE+IR IS OMITTED, AS IT IS IDENTICAL TO IE ON CLOSED-DOMAIN ATTRIBUTES. †EXCLUDED FROM MUCARS MACRO-AVERAGE, AS MILEAGE IS ENCODED AS RANGES.

Attribute	AutoScout24		MuCars	
	IE	*Full*	*IE*	*Full*
brand	0.992	1.000	1.000	1.000
model	0.783	0.865	0.862	0.930
year	1.000	1.000	1.000	1.000
mileage	1.000	1.000	0.332†	0.332†
engine_power	1.000	1.000	–	–
transmission_type	1.000	1.000	1.000	1.000
fuel_type	0.984	0.984	1.000	1.000
color	1.000	1.000	–	–
fuel_consumption	0.992	0.992	–	–
fuel_cons. unit	0.990	0.990	–	–
condition	–	–	0.998	0.998
doors	–	–	0.924	0.924
Macro-avg	**0.974**	**0.983**	**0.969**	**0.979**

F. Ablation

1) Cosine threshold τ for IE+IR: We evaluate $\tau \in \{0.50, 0.55, \ldots, 0.95\}$ on both evaluation datasets, re-using the IE extractions of Section IV-E so that only the retrieval-and-thresholding stage varies. Confidence bands are 95% percentile bootstrap intervals with 1,000 resamples (Fig. 3).

No single τ optimizes both quality metrics. Model macro F1 peaks at τ=0.90 on AutoScout24 (0.846) and at τ=0.85 on MuCars (0.870); novelty F1 instead peaks at $\tau \in [0.50, 0.65]$ on AutoScout24 (0.588) and at τ=0.80 on MuCars (0.795). Lowering τ filters IE noise from the predicted novel set, increasing novelty precision from 0.280 to 0.443 on AutoScout24 and from 0.520 to 0.839 on MuCars at τ=0.50, but the same aggressive matching collapses some genuine novelties into catalog entries, reducing novelty recall to 0.875 and 0.712. The resulting compression at the lowest τ ($\sim$13% on AutoScout24, $\sim$8% on MuCars) therefore mixes correct synonym collapses with false matches against true novelties. At every τ the variant remains below the full IE+IR+ER pipeline on both metrics (Table IV: model F1 0.890/0.885, novelty F1 0.623/0.878), whose compression of 4.2% and 5.6% concentrates on correct merges.

2) Retrieval depth k for IE+IR+ER: We evaluate $k \in \{1, 3, 5, 10, 20\}$ on both datasets, re-using the IE extractions of Section IV-E so that only the retrieval depth and entity-resolution loop vary. Confidence bands are 95% percentile bootstrap intervals with 1,000 resamples (Fig. 4).

Quality saturates by $k = 3$ to 5 on both datasets: the largest jump is from k=1 to k=3 (model F1 +3.3 percentage points on AutoScout24, +2.1 on MuCars), and beyond k=5 neither model F1 nor novelty F1 moves outside its bootstrap interval, except a slow drift in MuCars novelty F1 (0.884 at k=5 to 0.909 at k=20). Catalog compression on AutoScout24 spikes at k=1 (7.3%) before stabilizing near 5% for $k \geq 3$, indicating that with a single candidate the resolver tends to over-merge; once it has at least three candidates to compare against, it becomes more selective. MuCars compression varies by less than 1 percentage point across the grid. ER cost grows close to linearly with k above k=3, with k=20 requiring roughly twice the LLM calls per listing of k=5 on both datasets without quality return. The operating point k=5 used in Section IV-A therefore sits just past the quality saturation knee, where the resolver has enough candidates to avoid the k=1 over-merging regime while keeping cost bounded.

G. Downstream Utility

We train three gradient-boosting regressors (CatBoost, XGBoost, LightGBM) with default library hyperparameters to predict listing price on MuCars, averaging errors over ten random 20% holdout splits per model. The IE step is instantiated with a prompt variation in order to populate the schema in full: beyond extracting explicitly stated values, the LLM fills attributes a domain expert would derive with confidence from make, model, and year (e.g., body type, drive type, cylinders, engine aspiration, doors). We train each regressor on two configurations: the nine MuCars columns plus a 16-dimensional equipment one-hot encoding (*raw*, 25 features), and the same inputs concatenated with the 16 imputed schema fields (*normalized*, 41 features). The gap measures the marginal contribution of canonicalization plus imputation alone. Listings with implausible prices (outside 5,000 to 5,000,000 MAD) are excluded, leaving 75,733 rows. Missing numeric and categorical values are imputed on each fold by (brand, model) group median or mode with brand-level and overall fallbacks. The MuCars catalog is seeded from DVM-CAR (Section IV-A). On 21.6% of test rows the IE+IR+ER pipeline matched the raw model string to an existing canonical entry in the catalog and substituted the canonical label (e.g., `golf 7` to `golf`, `classe c` to `c`

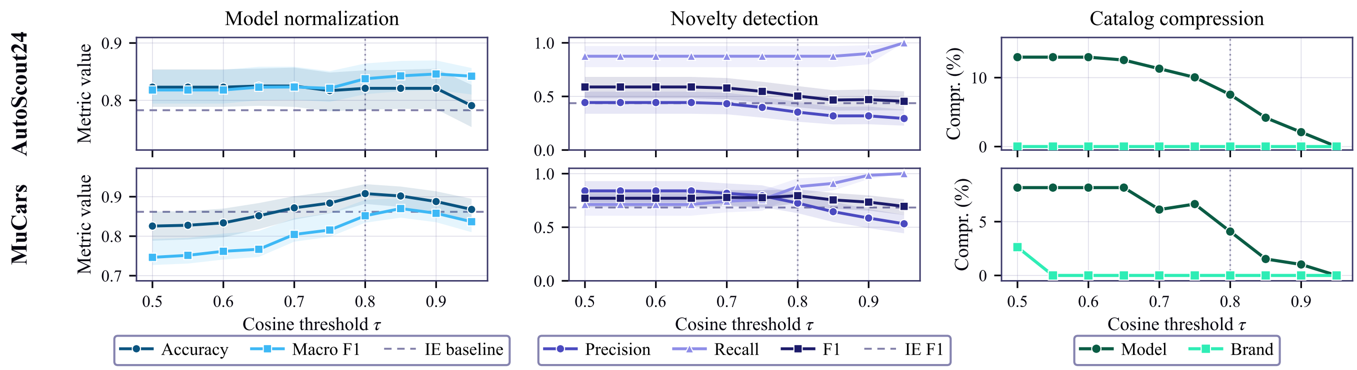

Fig. 3. IE+IR sensitivity to cosine threshold τ. Dashed: IE-only baseline; dotted: $\tau = 0.80$, used for the main results (Table IV).

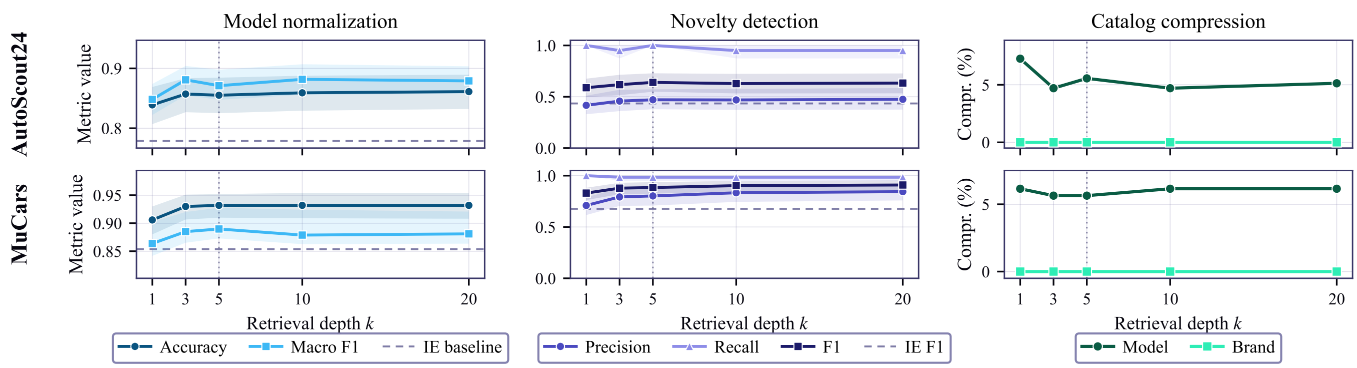

Fig. 4. IE+IR+ER sensitivity to retrieval depth k. Dashed: IE-only baseline; dotted: $k = 5$, used for the main results (Table IV).

`class`, `220` to `c class`); we call these the *canonicalized models*. The remaining 78.4% either match a catalog entry by surface form or are kept as novel entries with no surface change. We report results on the full test set and on the canonicalized-models subset separately.

Table VI reports the results. Normalization reduces error on every metric, every model, and both slices. On the full test set MAPE decreases by 2.7 to 5.3% (CatBoost weakest, XGBoost largest) and MAE by 4.4 to 6.4% (LightGBM weakest, XGBoost largest). On the canonicalized-models subset the reduction concentrates: MAPE decreases by 4.5 to 5.1%, MdAPE by 6.5 to 8.3%, and MAE by 5.5 to 8.7%, with CatBoost recording the largest MAE reduction at 8.7%. The improvement is consistent in direction and comparable in magnitude across architectures, indicating that the canonical labels carry signal that is robust to the choice of downstream estimator rather than an artefact of any one model's inductive bias.

H. Computational Cost and Scalability

We assess deployment feasibility by profiling the full IE+IR+ER pipeline on a stream of 10,000 records per dataset, processed with up to 50 concurrent requests, on a Mac Mini (Apple M4, 16 GB) under the configuration of Section IV-A. Fig. 5 reports how latency, throughput, and API calls evolve as the catalog grows.

a) Latency: Per-record latency is dominated by the extraction call and decreases as the catalog saturates and fewer records invoke entity resolution: the median is 3.8 s on AutoScout24 and 5.4 s on MuCars, with 95th percentiles of 8.4 and 12.7 s corresponding to records that still trigger resolution. This time is incurred almost entirely at the model API. Locally the pipeline performs little computation, running nearest-neighbor search on CPU in under 0.1 ms per record and using no GPU and at most 0.8 GB of memory regardless of corpus size.

b) Throughput: Throughput increases as the catalog saturates and resolution work disappears, reaching a steady 622 records per minute on AutoScout24 and 424 on MuCars from a single process. Because local resources are not the bottleneck, this rate is bounded by API throughput rather than by hardware, and scales with additional concurrency up to the provider's quota. The per-record cost amortizes likewise, to roughly \$0.37 and \$0.54 per thousand records. A complete pass over the AutoScout24 source (251k records) thus costs about \$92 and finishes in roughly seven hours on one process, and the MuCars source (102k records) about \$55 in about four

TABLE VI

DOWNSTREAM PRICE PREDICTION ON MUCARS OVER 10 RANDOM 20% HOLDOUT SPLITS; CELLS SHOW RAW → NORMALIZED (RELATIVE CHANGE).

Model	**MAPE**	**MdAPE**	**MAE** (MAD)
Full test set ($n = 15{,}147$)			
CatBoost	0.197 → **0.192** (−2.7%)	0.093 → **0.089** (−3.6%)	26,676 → **25,260** (−5.3%)
XGBoost	0.201 → **0.190** (−5.3%)	0.105 → **0.096** (−8.3%)	27,602 → **25,847** (−6.4%)
LightGBM	0.200 → **0.192** (−4.3%)	0.106 → **0.100** (−5.9%)	27,650 → **26,422** (−4.4%)
Canonicalized models ($n = 3{,}272$, 21.6% of test set)			
CatBoost	0.192 → **0.183** (−4.5%)	0.103 → **0.096** (−6.5%)	33,668 → **30,747** (−8.7%)
XGBoost	0.200 → **0.190** (−5.1%)	0.113 → **0.103** (−8.3%)	33,596 → **31,017** (−7.7%)
LightGBM	0.198 → **0.188** (−4.9%)	0.113 → **0.106** (−6.8%)	33,567 → **31,719** (−5.5%)

hours, with the runtime decreasing further under additional request parallelism.

c) API calls: Each record incurs exactly one extraction call. Only attribute values absent from the catalog require further work, namely an embedding for retrieval and one or more entity-resolution calls, while values already canonical require none. This added work stays small and diminishes as the catalog fills; entity resolution accounts for only 124 of the 10,124 calls on AutoScout24 and 1,523 of the 11,523 on MuCars, and embedding for a further 176 and 521. As the cache-hit rate climbs to 82% and 84%, the number of API calls per record falls toward the single extraction call (Fig. 5), settling at 1.00 on AutoScout24 and 1.06 on MuCars; the higher MuCars value reflects the greater variety of novel models in its Moroccan-market data, each of which must be resolved against retrieved candidates.

V. CONCLUSION

We presented a vehicle normalization framework that maps free-text listings to a 26-attribute schema by chaining LLM extraction, dense retrieval over self-growing per-attribute catalogs, and record-level LLM resolution. On 1,000 human-annotated listings from AutoScout24 and MuCars, the full pipeline improves model accuracy and novelty F1 over an extraction-only baseline on both datasets, and reduces price-prediction MAPE by 2.7–5.3% on MuCars across CatBoost, XGBoost, and LightGBM. The downstream improvement concentrates on the 21.6% of MuCars test rows whose model name was canonicalized by the pipeline: on this subset, MAE decreases by 5.5–8.7%, well above the full-set figure. Downstream regressors therefore benefit mostly from variant pooling: a single canonical label lets the regressor treat as one input what the raw data splits into separate categories.

Beyond predictive quality, the method exhibits favorable asymptotic behavior: as the knowledge catalog saturates, the per-record number of API calls converges to a single extraction call, keeping latency and cost bounded as the corpus grows. This makes its deployment feasible in large-scale production environments, as discussed in Section IV-H.

This study has three main limitations. First, open-domain coverage is narrow: only *brand* and *model* exercise the full pipeline, and brand normalization saturates because the seed catalog already covers nearly every brand. Second, evaluation is restricted to OpenAI models, leaving cross-family generalization untested. Third, the $n = 1{,}000$ test sample is small relative to comparable normalization benchmarks and yields wide bootstrap intervals on novelty F1.

Considering this, future work follows several directions. Expanding the open-domain attribute set and seeding the catalog from a union of markets would deepen vehicle taxonomy coverage and widen the canonicalized-models subset. Replicating the pipeline with open-source LLMs would test cross-family generalization Finally, extending the framework to general-purpose product normalization would enable evaluation on open-source benchmarks such as WDC-PAVE [19] and expand its applicability beyond vehicles.

ACKNOWLEDGMENT

This work was partially supported by the Agencia Nacional de Investigación y Desarrollo (ANID) under FONDECYT Initiation Project 11230351 and by CCTVal Basal FB0821.

REFERENCES

[1] Grand View Research, "Used car market size and share, Industry report, 2030," 2024, report ID GVR-4-68039-119-7. Accessed: 2026-04-23. [Online]. Available: https://www.grandviewresearch.com/industry-analysis/used-car-market

[2] S. Hu, S. X. Zhu, and K. Fu, "The manufacturer's resale strategy for trade-ins," *European Journal of Operational Research*, vol. 323, no. 2, pp. 583–598, 2025.

[3] N. Zacharof, J. Nur, D. Kourtesis, J. Krause, and G. Fontaras, "A review of the used car market in the european union," Luxembourg (Luxembourg), 2025.

[4] Zhang, Minshun, "Decoding consumer behavior in the used car market: A machine learning approach to key decision factors," *ITM Web Conf.*, vol. 70, p. 04033, 2025.

[5] International Organization for Standardization, "Road vehicles – vehicle identification number (vin) – content and structure," ISO 3779:2009, International Organization for Standardization, Geneva, Switzerland, 2009, accessed: December 8, 2025. [Online]. Available: https://www.iso.org/standard/52200.html

[6] National Highway Traffic Safety Administration, "Product information catalog and vehicle listing (vPIC)," 2025, accessed: 2025-04-06. [Online]. Available: https://vpic.nhtsa.dot.gov/

[7] Auto Care Association, "ACES and PIES data standards," 2026, ACES 5.0 and PIES 8.0 released March 2026. [Online]. Available: https://www.autocare.org/data-standards

[8] W3C Automotive Ontology Community Group, "Automotive extension—Schema.org," 2024, accessed: 2025-04-06. [Online]. Available: https://schema.org/docs/automotive.html

[9] COVESA, "Vehicle signal specification (VSS)," 2025, accessed: 2025-04-06. [Online]. Available: https://covesa.global/vehicle-signal-specification/

[10] ——, "Vehicle data model requirements and proposed solutions," 2025, accessed: 2025-04-06. [Online]. Available: https://wiki.covesa.global/display/WIK4/Vehicle+Data+Model+Requirements+and+Proposed+Solutions

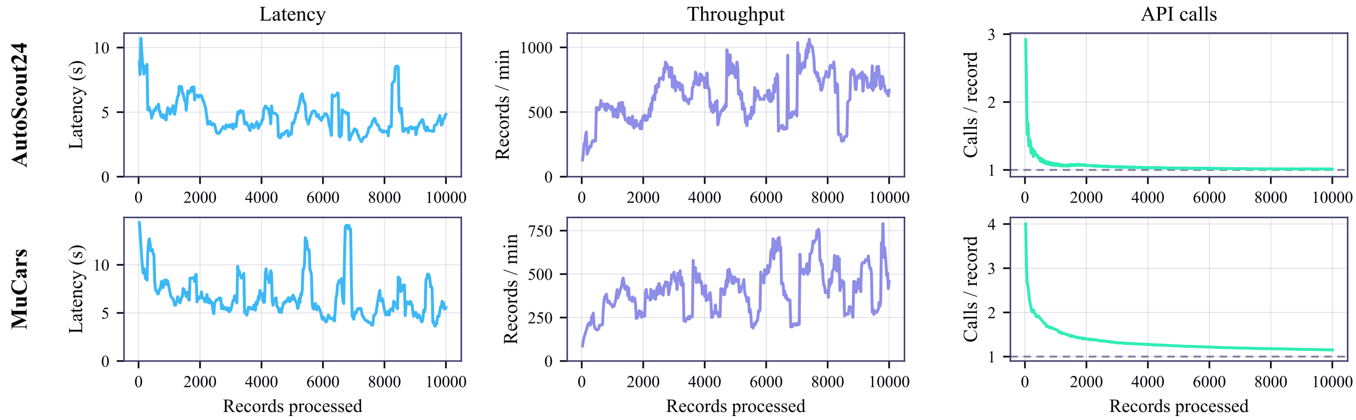

Fig. 5. Computational efficiency over a 10,000-record stream. Left: latency; center: throughput; right: API calls per record (dashed: extraction floor).

[11] A. Dutulescu, A. Catruna, S. Ruseti, D. Iorga, V. Ghita, L.-M. Neagu, and M. Dascalu, "Car price quotes driven by data-comprehensive predictions grounded in deep learning techniques," *Electronics*, vol. 12, no. 14, 2023.

[12] I. Fayyaz, G. G. M. N. Ali, and S. S. Khairunnesa, "Advanced feature engineering and machine learning techniques for high accurate price prediction of heterogeneous pre-own cars," *Vehicles*, vol. 7, no. 3, 2025.

[13] K. Madhusudhanan, G. Behrens, M. Stubbemann, and L. Schmidt-Thieme, "Probsaint: Probabilistic tabular regression for used car pricing," in *2024 IEEE International Conference on Big Data (BigData)*, 2024, pp. 2179–2187.

[14] S. Bergmann and S. Feuerriegel, "Machine learning for predicting used car resale prices using granular vehicle equipment information," *Expert Systems with Applications*, vol. 263, p. 125640, 2025.

[15] G. Zheng, S. Mukherjee, X. L. Dong, and F. Li, "OpenTag: Open attribute value extraction from product profiles," in *Proc. ACM SIGKDD*, 2018, pp. 1049–1058.

[16] Q. Wang, L. Yang, B. Kanagal, S. Sanghai, D. Sivakumar, B. Shu, Z. Yu, and J. Elsas, "Learning to extract attribute value from product via question answering: A multi-task approach," in *Proc. ACM SIGKDD*, 2020, pp. 47–55.

[17] A. Brinkmann, R. Shraga, and C. Bizer, "Product information extraction using ChatGPT," *arXiv preprint arXiv:2306.14921*, 2023.

[18] ——, "ExtractGPT: Exploring the potential of large language models for product attribute value extraction," in *Proc. iiWAS*, ser. LNCS, vol. 15342. Springer, 2024.

[19] A. Brinkmann, R. Baumann, and C. Bizer, "Using LLMs for the extraction and normalization of product attribute values," in *Proc. ADBIS*, ser. LNCS, vol. 14918. Springer, 2024, pp. 217–230.

[20] O. Sainz, I. García-Ferrero, R. Agerri, O. L. de Lacalle, G. Rigau, and E. Agirre, "GoLLIE: Annotation guidelines improve zero-shot information-extraction," in *Proc. ICLR*, 2024.

[21] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. tau Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. NeurIPS*, 2020, pp. 9459–9474.

[22] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. tau Yih, "Dense passage retrieval for open-domain question answering," in *Proc. EMNLP*, 2020, pp. 6769–6781.

[23] G. Izacard, M. Caron, L. Hosseini, S. Riedel, P. Bojanowski, A. Joulin, and E. Grave, "Unsupervised dense information retrieval with contrastive learning," *Transactions on Machine Learning Research*, 2022.

[24] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2021.

[25] M. Douze, A. Guzhva, C. Deng, J. Johnson, G. Szilvasy, P.-E. Mazaré, M. Lomeli, L. Hosseini, and H. Jégou, "The Faiss library," *arXiv preprint arXiv:2401.08281*, 2024.

[26] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using siamese BERT-networks," in *Proc. EMNLP-IJCNLP*, 2019, pp. 3982–3992.

[27] L. Wang, N. Yang, X. Huang, B. Jiao, L. Yang, D. Jiang, R. Majumder, and F. Wei, "Text embeddings by weakly-supervised contrastive pre-training," *arXiv preprint arXiv:2212.03533*, 2022.

[28] S. Xiao, Z. Liu, P. Zhang, and N. Muennighoff, "C-Pack: Packed resources for general chinese embeddings," in *Proc. ACM SIGIR*, 2024.

[29] Y. Li, J. Li, Y. Suhara, A. Doan, and W.-C. Tan, "Deep entity matching with pre-trained language models," *Proc. VLDB Endowment*, vol. 14, no. 1, pp. 50–60, 2020.

[30] A. Narayan, I. Chami, L. Orr, and C. Ré, "Can foundation models wrangle your data?" *Proc. VLDB Endowment*, vol. 16, no. 4, pp. 738–746, 2022.

[31] R. Peeters and C. Bizer, "Using ChatGPT for entity matching," in *VLDB Workshop on AI & DM*, ser. LNCS. Springer, 2023.

[32] R. Peeters, R. Steiner, and C. Bizer, "Entity matching using large language models," in *Proc. EDBT*, 2025, pp. 529–541.

[33] R. Steiner, R. Peeters, and C. Bizer, "Fine-tuning large language models for entity matching," *arXiv preprint arXiv:2409.08185*, 2024.

[34] Z. Zhang, A. Goswami, D. Bespalov, and Y. Li, "AnyMatch: Efficient zero-shot entity matching with a small language model," *arXiv preprint arXiv:2409.04073*, 2024.

[35] T. Wang, H. Lin, X. Han, L. Sun, and X. Chen, "Match, compare, or select? An investigation of large language models for entity matching," in *Proc. COLING*, 2025, pp. 96–109.

[36] J. Huang, B. Chen, L. Luo, S. Yue, and I. Ounis, "Dvm-car: A large-scale automotive dataset for visual marketing research and applications," in *2022 IEEE International Conference on Big Data (Big Data)*, 2022, pp. 4140–4147.

[37] E. Tabarnoust, M. Mghari, and Y. Zaz, "Moroccan used cars dataset: Insights into the used car market," *Data in Brief*, vol. 63, 2025.

[38] ——, "Moroccan used cars dataset (MUCars - 2024)," 2024, accessed: 2025-04-06.

[39] wspirat, "Germany used cars dataset 2023," https://www.kaggle.com/datasets/wspirat/germany-used-cars-dataset-2023, 2023, license: CC0 Public Domain. Accessed: 2026-04-20.

[40] J. L. Fleiss, "Measuring nominal scale agreement among many raters," *Psychological Bulletin*, vol. 76, no. 5, pp. 378–382, 1971.

[41] J. R. Landis and G. G. Koch, "The measurement of observer agreement for categorical data," *Biometrics*, vol. 33, no. 1, pp. 159–174, 1977.

[42] R. Artstein and M. Poesio, "Inter-coder agreement for computational linguistics," *Computational Linguistics*, vol. 34, no. 4, pp. 555–596, 2008.